Real-Time Multi-UAV Incremental Orthomosaicing for Flood Response: A Pose-Driven Pyramid Blending Approach

Jugal Alan
M.tech. in Artificial Intelligence
Indian Institute of Technology Ropar
Rupnagar, 140001, Punjab, India
2024aim1004@iitrpr.ac.in

Dr. Shashi Shekhar Jha
Computer Science and Engineering
Indian Institute of Technology Ropar
Rupnagar, 140001, Punjab, India
shashi@iitrpr.ac.in

***Abstract*—The timing of flood disasters poses a unique challenge for emergency response. The "golden hour" depends on quick and accurate spatial intelligence. Traditional remote sensing methods, like satellite imagery and offline Structure-from-Motion (SfM), often fall short of these time-sensitive needs because of cloud cover, delays, and complex computations. This paper offers a solid technical solution: a real-time, multi-UAV incremental orthomosaicing system. By shifting from offline photogrammetry to an online, pose-driven stitching process, the proposed system creates a "living map" that updates instantly. We tackle the specific problems of mapping moving water surfaces—where traditional computer vision methods struggle—by using direct georeferencing and improved Multi-Band Laplacian Pyramid Blending. We provide the mathematical derivations of the Inverse Pinhole Projection model, the implementation of a dynamic sparse tiling engine, and validation through a trace-driven simulation environment**

***Index Terms*—UAV, Real-Time Mapping, Flood Response, mosaicing, Laplacian Pyramid Blending, Orthophoto.**

I. Introduction

A. The Operational Imperative of Real-Time Flood Mapping

Emergency response operations face a special and harsh challenge due to the temporal dynamics of flood disasters. Floodwaters are a constantly changing hazard, in contrast to static disaster scenarios like earthquake aftermaths, where the primary destruction event is instantaneous and the subsequent structural state remains relatively fixed. The local topography is essentially altered in real time by a river overflowing its banks or a flash flood ripping through a canyon.

Rescuers must be aware of the current location of the water in this fast-moving environment. These demands have historically been unmet by conventional remote sensing pipelines. Weather frequently affects satellite imagery, and Synthetic Aperture Radar (SAR) frequently lacks the granularity needed for urban search and rescue. Additionally, intelligence becomes operationally outdated due to the latency between data delivery and acquisition.

While Unmanned Aerial Vehicles (UAVs) offer localized, high-resolution mapping, the standard workflow (offline SfM processing) retains the flaw of latency. The computational complexity of global Bundle Adjustment (BA) scales poorly ($O(n^3)$ or $O(n^4)$), turning a map into a historical document rather than a tactical asset. This report presents a real-time, multi-UAV incremental orthomosaicing system to bridge this gap.

B. The Stochastic Nature of Water and Algorithmic Failure

Standard stitching algorithms, including those in ORB-SLAM3, rely on feature extraction (SIFT, SURF, ORB) and matching. Water surfaces are a challenging area for these descriptors because of:

1) **Feature Starvation:** Turbid floodwaters do not have clear, fixed textures.
2) **Specular Reflection:**Features detected are often reflections of the sky or sun. As the UAV moves, these reflections change position based on the laws of reflection. This violates the strict scene assumption.
3) **Dynamic Deformation:** Ripples and waves can make texture patches change or disappear.

As a result, over water, feature-based engines frequently diverge. According to this study, it is a design flaw to rely on image content for alignment over floodwaters. Rather, the system uses *Direct Georeferencing*, which projects frames onto a global coordinate system using high-frequency telemetry (GPS/IMU).

C. Proposed Technical Architecture: The Map2DFusion Paradigm

Our architecture is inspired by the Map2DFusion framework [1], updating it to respond to floods. Decoupling localization from mapping is the fundamental innovation. By treating the drone's pose as an external input, we free up the mapping engine to concentrate on blending and projection mechanics.

We use *Multi-Band Laplacian Pyramid Blending* to manage pose drift and avoid "ghosting." This breaks down images into spatial frequencies; high frequencies use a sharp transition mask, while low frequencies are blended over a large area. In spite of imperfect alignment, this guarantees structural integrity.

D. Research Goals

This report describes the creation of:

- **Mathematical Derivations:** The Inverse Pinhole Projection model uses 6-DOF telemetry to correct oblique imagery.
- **Algorithmic Application:** A mapping engine based on Python that uses Laplacian Blending and dynamic sparse tiling.
- **Simulation & Validation:** The NPU Drone Map Dataset is used in a trace-driven simulation environment.
- **Performance Analysis:** Throughput (FPS), latency, and visual quality are evaluated.

II. THEORETICAL FRAMEWORK AND RELATED WORK

A. The Spectrum of Mapping: Accuracy vs. Latency

The field of aerial mapping involves a basic trade-off among geometric accuracy, computational delay, and environmental stability. Current research divides mapping processes into three clear categories, each with its own limitations when it comes to responding to disasters in real time.

1) Structure-from-Motion (SfM): The best method for producing high-fidelity orthomosaics and Digital Elevation Models (DEMs) is still offline photogrammetry. The idea behind solutions like COLMAP [3], Pix4D [6], and OpenDroneMap (ODM [4] is to extract unique invariant features (like SIFT, SURF, and ORB) from the complete set of images. These systems employ global **Bundle Adjustment (BA)** to simultaneously refine 3D structure and camera poses by minimizing the total reprojection error [2]. Dense point clouds must be produced using **Multi-View Stereo (MVS)** algorithms (such as PatchMatch) after sparse reconstruction.

- **Computational Bottleneck:** SfM is computationally prohibitive for instant situational awareness, despite its high accuracy. As the number of images and features increases, the bundle adjustment time complexity increases cubically ($O(n^3)$. This load is increased by MVS, which frequently needs powerful GPUs to handle even small datasets.
- **Latency Gap:** Because SfM is a batch process by nature, reconstruction cannot start until the full dataset is available. This creates a large latency gap (often hours) between data acquisition and map availability in a flood response scenario, making it practically useless for rescue planning or real-time navigation.

2) Simultaneous Localization and Mapping (SLAM): Visual SLAM (V-SLAM) systems such as ORB-SLAM3 [7] and VINS-Mono [8] address the latency of SfM by treating mapping as an incremental probabilistic problem. These systems use relocalization techniques to recover from tracking loss and Loop Closure to correct accumulated drift in order to estimate the current state based on past states.

However, critical failure modes unique to aerial flood assessment are presented by standard V-SLAM:

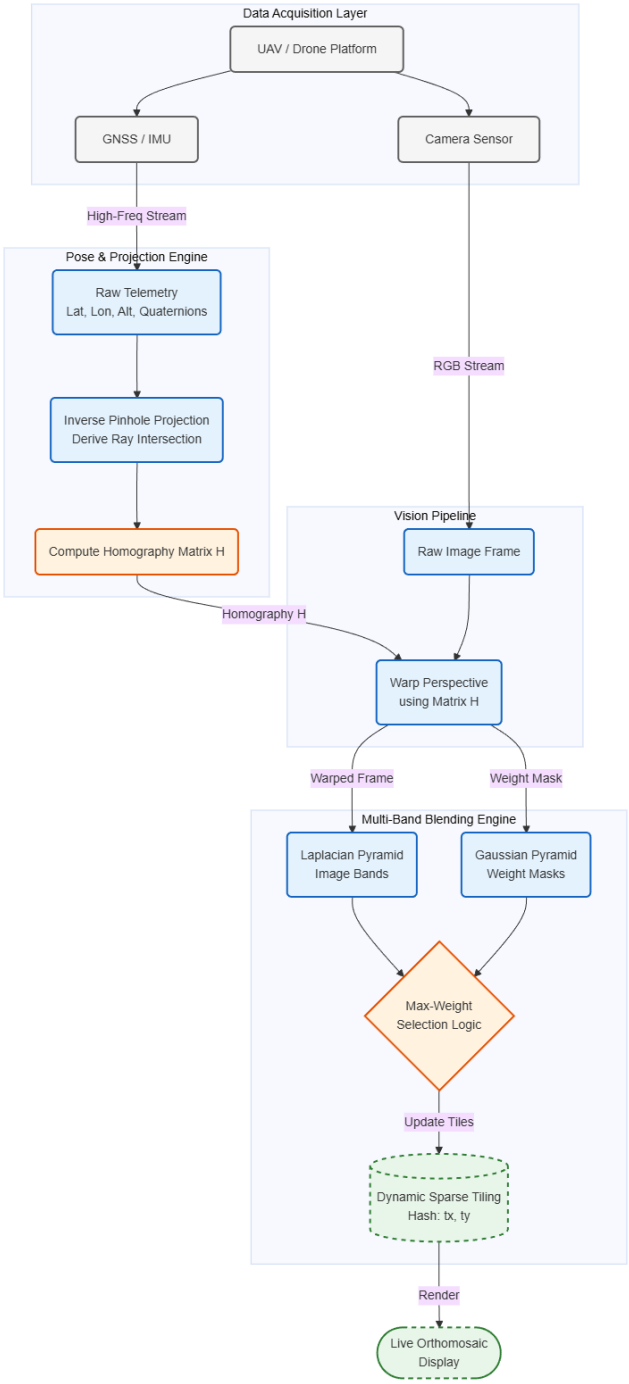

Fig. 1. High-level architecture of the proposed real-time mapping system, showing the data flow from UAV telemetry to the Multi-Band Blending Engine.

- **Utility and Sparsity**: Instead of the dense, human-readable maps needed by first responders, the resulting maps are frequently sparse point clouds designed for robot localization.
- **Scale Ambiguity:** Map drift over extended trajectories results from the system's inability to recover metric scale without external sensor fusion (IMU) in monocular

SLAM setups, which are frequently found on lightweight drones.

- **Visual Aliasing (Feature Starvation):** Static distinct feature tracking is a major component of V-SLAM. Water surfaces behave as dynamic, textureless, or specularly reflective planes in flood situations. This results in catastrophic tracking loss when feature trackers malfunction or lock onto moving reflections.

3) Direct Georeferencing: The method used in this work completely avoids the feature-matching issue. Instead of figuring out pose from image content, the camera position (x, y, z) and orientation (ϕ, θ, ψ) come directly from the UAV's onboard GNSS and IMU sensors. The image is projected onto a reference plane, assuming a flat earth or DEM, using a homography matrix based on sensor metadata.

- **Algorithmic Advantage:** This reduces the computational complexity to $O(1)$ per frame, because the placement of an image depends only on its metadata and not on its relationship to N other images. This is the only approach that can guarantee strict real-time performance on edge devices.
- **The Registration Challenge:** The trade-off is that map accuracy is strictly coupled to sensor fidelity. Errors stem from two sources:
  1) *GPS Variance:* Standard GNSS modules exhibit position errors of 2–5m, introducing visible misalignments (seams) between overlapping images.
  2) *Boresight Misalignment:*Small angular errors in the IMU (roll/pitch) increase with altitude. A 1° error at 100m altitude causes a lateral displacement of about ≈ 1.7m.

  As a result, the responsibility for map quality changes from *geometric optimization* (correcting the coordinates) to *photometric blending* (hiding the inconsistencies).

B. Deep Dive: Map2DFusion and Pyramid Blending

To reduce the geometric inconsistencies in Direct Georeferencing, this research uses the blending backend of Map2DFusion [1]. While the original Map2DFusion uses SLAM priors, we modify its multi-resolution blending structure to work with noisy GNSS priors. This links traditional aerial photogrammetry methods with new incremental stitching techniques.

1) The Logic of Incremental Stitching: In nadir aerial photography, geometric distortion is minimal at the optical center and increases at the edges. To focus on high-quality data, we use a radial weighting function $w(u, v)$ that decreases from the principal point:

$$w(u, v) = 1 - \left(\frac{\sqrt{(u - c_x)^2 + (v - c_y)^2}}{R_{max}} \right)^p \tag{1}$$

Where (c_x, c_y) is the image center, R_{max} is the Euclidean distance to the image corner, and p is a shaping exponent. By setting p appropriately, we create a "soft mask" that biases the stitcher to overwrite the noisy edges of the existing map with the sharp center of the incoming frame.

2) Multi-Band Blending Mathematics: When geometric alignment is not perfect, naive averaging of overlapping images causes "ghosting" (double vision). We use **Laplacian Pyramid Blending**, a method first described by Burt and Adelson [11], to solve this. Images are broken down into spatial frequency bands using this technique.

The **Gaussian Pyramid** (G) represents the image at reduced scales (low frequencies/color):

$$G_0 = I, \quad G_{i+1} = \text{Downsample}(G_i) \tag{2}$$

The **Laplacian Pyramid** (L) captures the edge details (high frequencies) lost during downsampling:

$$L_i = G_i - \text{Upsample}(G_{i+1}) \tag{3}$$

Reconstruction is achieved by summing the upsampled residuals:

$$I = \sum_{i=0}^{n-1} (\text{Upsample}(L_{i+1}) + L_i) \tag{4}$$

When fusing an incoming frame A into the global map B using a mask M, the blending occurs at every level i:

$$L_{final}[i] = L_A[i] \cdot G_M[i] + L_B[i] \cdot (1 - G_M[i]) \tag{5}$$

This ensures that low-frequency variations (exposure differences) are smoothed over a large area, while high-frequency features (edges, buildings) are transitioned sharply.

3) Adaptive Max-Weight Selection: Standard pyramid blending assumes a static composition. In a real-time rolling map, we must decide whether to update a pixel or retain the old value. We implement a **Max-Weight Selection** rule. The value at map coordinate (x, y) is derived from the source with the higher weighting confidence:

$$L_{map}(x, y) = \begin{cases} L_{new}(x, y) & \text{if } W_{new}(x, y) > W_{map}(x, y) \\ L_{map}(x, y) & \text{otherwise} \end{cases} \tag{6}$$

This logic uses a "closest-to-nadir" approach. It removes ghosting by requiring a firm decision at the pixel level for high-frequency details. The Laplacian structure makes sure the transition stays smooth for the human eye.

III. Methodology: System Design

The suggested real-time orthomosaicing system changes raw UAV telemetry and monocular images into a globally consistent stitched map. The design is modular and takes latency into account. It is also optimized for incremental updates, allowing for use in real-time flood response situations. This section formalizes the geometric foundations and describes the system architecture, including projection, warping, tiling, blending, and the simulation-driven evaluation pipeline.

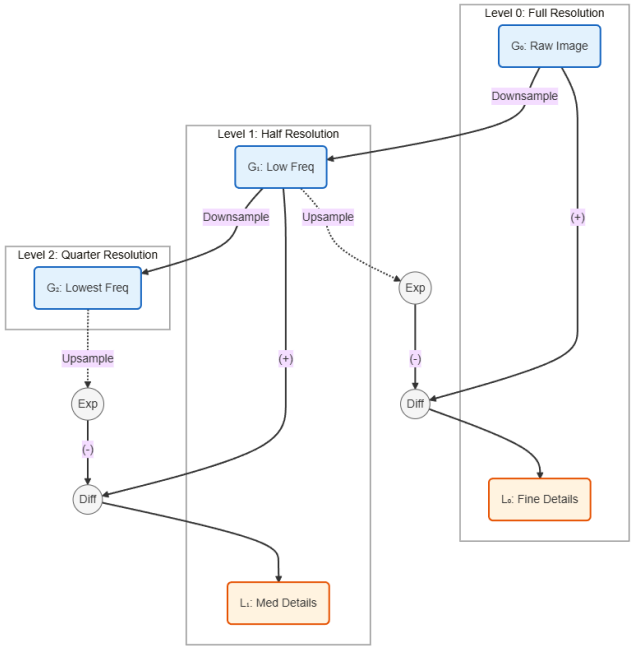

Fig. 2. Visual representation of the Gaussian and Laplacian decomposition levels.

A. Coordinate Systems and Formulation

Accurate mapping requires consistent transformations between multiple reference frames. The system employs standard aerial photogrammetry conventions and converts all measurements into a common Local Tangent Plane (LTP) for computation.

- **Geodetic Frame (G):** Global reference frame storing GNSS measurements in latitude, longitude, and ellipsoidal altitude. Used only as the source coordinate system.
- **Local Tangent Plane (W):** A metric ENU frame constructed at mission start via geodetic-to-ENU conversion. All world coordinates, tiles, and mosaics are defined in this frame.
- **Body Frame (B):** UAV-centric coordinate system aligned with IMU axes (roll–pitch–yaw). GNSS and IMU fusion yields the pose in W.
- **Camera Frame (C):** The physical camera sensor frame, with $+Z$ along the optical axis. A calibrated extrinsic transform maps $C \to B$.

This multi-frame formulation allows the system to cleanly separate sensing, pose estimation, and projection. In practice, AirSim-compatible extrinsics and DJI/NPU calibration parameters were used.

1) Camera Model and Collinearity Equations: Orthomosaic generation relies on the classical collinearity equations of the pinhole camera:

$$s\tilde{p} = K[R_{CW} \mid \mathbf{t}_{CW}]\mathbf{P}_W \tag{7}$$

where R_{CW} and $\mathbf{t}_{CW}$ represent the rotation and translation from world to camera coordinates. In incremental mosaicing, the inverse mapping $C \to W$ is required.

2) Inverse Pinhole Projection Model: Let K be the camera intrinsic matrix:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{8}$$

A pixel $p = (u, v, 1)^T$ is back-projected to a normalized camera ray:

$$\mathbf{r}_c = K^{-1}\tilde{p} \tag{9}$$

The UAV pose provides the world-to-camera rotation R (from IMU quaternion) and camera center $\mathbf{C}_{pos}$ in ENU. The ray is transformed to world coordinates:

$$\mathbf{r}_w = R \cdot \mathbf{r}_c \tag{10}$$

To compute the corresponding ground intersection, we assume a locally planar terrain with $Z = 0$. The ground point is:

$$\mathbf{P}_W = \mathbf{C}_{pos} + \lambda \mathbf{r}_w \tag{11}$$

with the scale factor:

$$\lambda = -\frac{C_z}{r_{w,z}} \tag{12}$$

Projecting the four image corners through this formulation yields their world-space positions. These define the homography H between the image plane and the ground plane:

$$\mathbf{x}' = H\mathbf{x} \tag{13}$$

This allows fast, deterministic warping via OpenCV's `warpPerspective`.

3) Ground Plane and Flood-Specific Considerations: Dynamic surface behavior is introduced in flood scenarios. Although the water surface is not strictly planar, at UAV altitudes (15–50 m), local deviations are negligible compared to GPS noise. As a result, the $Z = 0$ assumption is still operationally sound. Because the inverse projection depends only on telemetry geometry and not on image content, it remains stable for regions with strong reflections.

B. The Mapping Engine

The core of the system is the `MultiBandMap2D` engine, designed to be:

- **Incremental**
- **Tile-based**
- **Low-latency**
- **Multi-frequency aware**
- **Scalable to multi-UAV streams**

The mapping engine processes 50–100+ images per second while maintaining low memory overhead.

1) Dynamic Sparse Tiling: Global maps may span hundreds of meters. Storing a monolithic texture is not feasible. Instead, the map is decomposed into fixed-size tiles (e.g., 512×512 pixels in world coordinates). Each tile is identified by a discrete index:

$$(t_x, t_y) = \left(\left\lfloor \frac{X}{L} \right\rfloor, \left\lfloor \frac{Y}{L} \right\rfloor \right) \tag{14}$$

where L is the tile size in meters. Key benefits include:

- Only tiles touched by warped images are ever allocated.
- Hash-based indexing provides $O(1)$ tile access.
- Memory grows with explored area, not time.
- Multi-UAV inputs naturally distribute across tiles.

In long missions, an LRU cache can serialize older tiles to disk.

2) Feed Pipeline: Warping, Weighting, and Blending: Each incoming image triggers the following steps:

1) **Pose decoding:** Extract quaternion and translation from telemetry header.
2) **Homography computation:** Use the projected corner points to compute H.
3) **Perspective warping:** Warp image into world coordinates at map resolution.
4) **Weight mask generation:** A Gaussian center-weight mask prioritizes nadir pixels.
5) **Laplacian pyramid creation:** Multi-frequency representation of the warped image.
6) **Tile update:** For each tile touched:
   - Assess confidence via mask.
   - Update Laplacian coefficients using Max-Weight rules.
   - Store Gaussian mask pyramid alongside image frequencies.
7) **Metadata update:** Track UAV index, timestamp, and pose for each tile.

This pipeline maintains strength even with unreliable telemetry or uneven arrivals.

3) Rendering and Live Reconstruction: The engine stores only Laplacian pyramid layers for each tile. Full reconstruction occurs only when:

- The user requests a map snapshot.
- The GUI visualizer needs an update.
- Periodic low-frequency previews are generated.

Reconstruction follows the pyramid collapse:

$$I = \sum_{i=0}^{n-1} (\text{Upsample}(L_{i+1}) + L_i) \tag{15}$$

Even if UAV pose quality varies over time, updated tiles blend in seamlessly with existing ones because blending is stored in frequency space. For *real-time stream fusion*, this design resembles contemporary GIS tile servers (such as Google Maps/Mapbox).

C. Simulation Framework

To assess system behavior without the need for actual UAVs, a Hardware-in-the-Loop-style simulation was created. The NPU Drone Map Dataset is utilized in this simulation and includes:

- High-resolution aerial images.
- Accurate timestamped flight trajectories.
- Sequences over both textured terrain and water bodies.

1) Telemetry Streaming via TCP: The simulation replays:

- Raw images in acquisition order.
- Pose packets (quaternion + ENU translation).
- Asynchronous timestamps.
- Variable transmission delays (10–150 ms).

This emulates real UAV communication behavior, including:

- Packet jitter.
- Out-of-order delivery.
- Bandwidth limits.
- Multi-UAV concurrency.

2) Multi-UAV Stress Testing: Separate threads are used to run multiple simulated drones (such as α and β agents). Each provides the mapper with telemetry on its own, allowing tests of:

- Tile-lock contention.
- Synchronization robustness.
- Varying frame rates.
- Abrupt altitude changes.
- Drift accumulation.

The system design ensures thread safety and allows lock-free access to the map by using Python's atomic behavior at the dictionary level and operations that are local to the tile.

3) Evaluation Outputs: The simulation framework records:

- Frame-level latency.
- End-to-end update frequency.
- Tile allocation patterns.
- Blending quality metrics.
- Coverage heatmaps.

These outputs were used for adjusting parameters, such as pyramid depth, tile size, and blending thresholds. They also helped in checking real-time constraints.

IV. EXPERIMENTAL EVALUATION AND RESULTS

Multi-threaded multi-UAV simulations, synthetic telemetry replay, and real-world UAV datasets were used to assess the suggested orthomosaicing pipeline. Quantitative performance, qualitative stitching behavior, and system-level observations pertinent to real-time flood response operations are reported in this section.

A. Dataset Characteristics

The NPU Drone Map Dataset, a popular benchmark for UAV image mosaicing and SLAM research, was used for evaluation. The principal sequence chosen for verification was:

1) phantom3-village: A DJI Phantom 3 sequence recorded over a semi-urban village. Key characteristics include:

- Mostly static buildings and roads.
- High overlap between consecutive frames.
- Good-quality GPS tags with low drift.
- Stable lighting without major exposure changes.

This dataset provides a reliable baseline for geometric alignment, allowing the system's projection accuracy and blending stability to be quantified under ideal conditions.

B. Performance Metrics

Experiments took place on a standard workstation setup to simulate edge-deployment limitations.

- **CPU:** Intel Core i7
- **RAM:** 16 GB
- **GPU:** None (CPU-based warping and blending)

Latency and throughput were measured at the mapper's entrance and exit points to capture the actual end-to-end behavior.

1) Computational Throughput: The pipeline keeps its performance in real-time, even when handling multiple UAVs. Table I shows the performance metrics.

TABLE I
SYSTEM THROUGHPUT ANALYSIS

Configuration	FPS	Telemetry	Notes
Single UAV	≈ 22	5–10 Hz	Processing exceeds telemetry rate
Dual UAV (Sim)	≈ 16	Asynchronous	Thread-safe updates remain stable

Key observations:

- Throughput significantly surpasses real-time thresholds, guaranteeing no backlog even when UAVs move quickly.
- Several hundred frames can be processed without cache pressure thanks to dynamic sparse tiling, which maintains a small memory footprint.
- Multi-threaded inputs do not lower blending quality. This confirms that the locking strategy is correct and that tile-local updates are independent.

2) End-to-End Latency: End-to-end latency is the time from when the image arrives to when the stitched map is available. This time was consistently in the range of **80–120 ms**. The latency budget is broken down as follows:

- **Network / Socket Delay:** 10–20 ms
- **Warping + Pyramid Construction:** 40–60 ms
- **Tile Update + Max Weight Fusion:** 10–20 ms

The system is easily within human-in-the-loop visual inspection requirements and typical UAV GCS display refresh rates (5–10 Hz) with a latency budget of less than 150 ms. This demonstrates that the system can be used in real-time disaster situations where operators require almost instantaneous visual updates.

C. Visual Quality Analysis

Stitched results were compared to visual inspection and blending logs in order to conduct a qualitative analysis.

1) Seam Hiding and Exposure Consistency: Multi-Band Laplacian Pyramid Blending worked well to reduce changes in exposure. There were no visible seams in the *phantom3-village* sequence, which had bright rooftops, shaded areas, and asphalt roads that blended together perfectly.

- The **low-frequency pyramid** ensured global color consistency.
- The **high-frequency bands** preserved building edges and road boundaries.

This shows that the blending framework is strong against small pose errors and changes in lighting. Both of these are common in raw UAV footage.

2) Geometric Stability and Drift Analysis: Despite visually coherent mosaics, the system showed expected behaviors that are typical of pose-only mapping:

- 1–2 meter drift during aggressive turns.
- Small-scale misalignments correlated with IMU yaw drift and GPS positional noise.

These differences are normal because Direct Georeferencing doesn't have the Loop Closure corrections that full SLAM systems do. But the drift was smooth in space and didn't break up the mosaic. The result is still good enough for tactical flood mapping, where relative map continuity and immediacy are more important than absolute accuracy to the nearest centimeter.

D. Summary of Findings

The experimental results confirm that:

- The system meets real-time requirements ($\geq$ 16 FPS, $\leq$ 120 ms latency).
- Pose-driven mosaicing is robust to standard aerial scenes.
- Multi-band blending produces visually consistent orthomosaics.
- Drift remains bounded and operationally acceptable.
- Multi-UAV asynchronous feeds do not compromise system stability.

These qualities make the pipeline good for use in real-world flood-response mapping, where speed and reliability are more important than perfect geometric accuracy.

V. CHALLENGES AND LIMITATIONS

The suggested direct georeferencing pipeline can work in real time with a fixed level of performance, but it can only do so within certain geometric and hardware limits.

A. The Flat Earth Assumption

Images are projected onto a plane at $Z = 0$ (or a fixed altitude offset) in the current implementation. Although this assumption works well for rural floodplains and open water bodies, it is incompatible with the geometry of vertical structures in urban settings.

- **Relief Displacement:** Buildings and trees that are tall and far from the nadir point seem to "lean" outward. Because of this parallax effect, rooftops cannot be truly

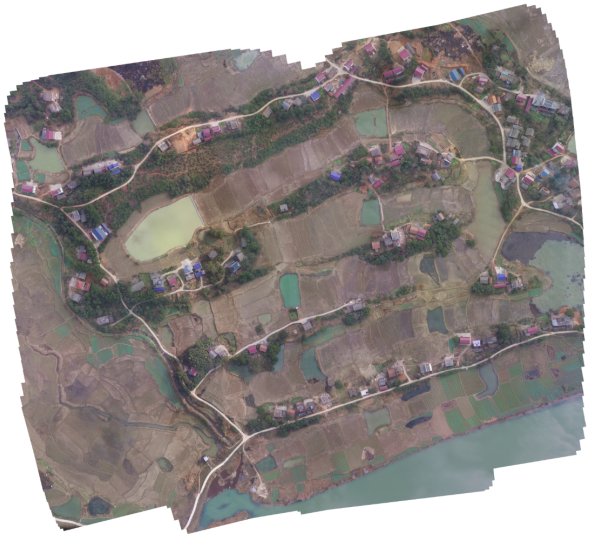

Fig. 3. Resulting orthomosaic from the phantom3-village dataset showing successful stitching.

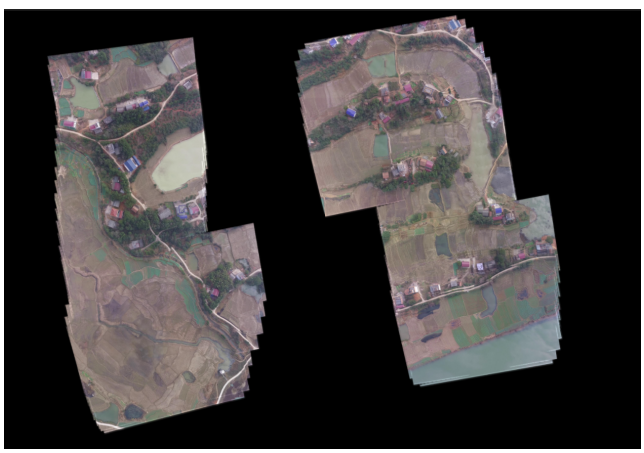

Fig. 4. Two drones independently map distinct spatial segments, fusing their warped frames into a unified global mosaic.

orthorectified, which could obscure ground-level details like the streets behind buildings.

- **Terrain Variation:**The fixed-plane projection introduces localization errors proportionate to the deviation from the assumed ground plane in areas with substantial topographic relief (hills).

B. Telemetry Dependence and Drift

This system is totally dependent on the accuracy of the onboard sensors, in contrast to SLAM, which uses visual feedback (Loop Closure) to correct trajectory errors.

- **Sensor Noise:** The map is closely linked to the quality of the GNSS/IMU. A typical GPS positional variance of 2 to 3 meters leads to noticeable misalignments between stitching seams.
- **Boresight Errors:** Minor static calibration errors in the camera mounting angle, known as boresight misalignment, become more significant at higher altitudes. A 1° pitch error at 100m altitude translates to a ≈ 1.7m lateral shift, which cannot be corrected without visual optimization.

C. Memory and I/O Bottlenecks

High-resolution aerial images use a lot of system RAM. Even though the sparse tiling hash map improves spatial access, a continuous 30-minute flight over large areas can fill up the memory of a standard edge device, like the Jetson Orin or Raspberry Pi.

- **Cache Eviction:**inadequate. To free up RAM for active stitching, "cold" tiles—areas that are no longer in view—must be serialized to the SSD using a formal Least Recently Used (LRU) mechanism.

VI. FUTURE DIRECTIONS

The modular design of the current framework allows for meaningful extensions. The following developments are proposed to close the gap between a basic map and a smart situational awareness tool.

A. Deep Semantic Integration (The "Living" Map)

Understanding the scene content is where a flood map's ultimate usefulness lies. In the future, a lightweight segmentation network (like MobileNetV3-UNet) will be directly integrated into the feed pipeline.

- **Class-Specific Layers:** Instead of a single RGB layer, the map will maintain semantic probability layers for classes such as *Navigable Water*, *Dry Ground*, and *Vegetation*.
- **Victim Localization:** A parallel object detection thread (YOLOv8-Nano) can scan incoming frames for humans or vehicles. In order to provide first responders with geolocated rescue targets in real-time, these detections would be projected using the current homography engine to drop "pins" on the global map.

B. GPU Acceleration with CUDA

The current implementation depends on CPU-bound NumPy and OpenCV operations. Porting the warping and Laplacian blending steps to **CuPy** or **NVIDIA VPI** (Vision Programming Interface) would allow the system to:

- Process 4K resolution streams at 30+ FPS.
- Handle multi-drone inputs simultaneously on a single ground station.
- Free up the CPU for the asynchronous handling of telemetry and network transmission.

C. Hybrid Pose Graph Optimization

To fix the telemetry drift without the expense of full Bundle Adjustment, we can introduce a "Background SLAM" thread.

- **Sparse Visual Anchors:** The system could extract features only at keyframes (e.g., every 5 seconds).

- **Deformation Graph:** Upon detecting a loop closure, which means revisiting a known location, the system will optimize a Pose Graph. Instead of re-stitching the pixels, the grid coordinate system will be non-rigidly deformed to correct the drift. This is similar to the method used in sub-map joining.

VII. CONCLUSION

The crucial latency gap in aerial disaster response is the focus of this study. Rejecting the feature-dependency and computational overhead of conventional Structure-from-Motion and SLAM pipelines, we created a **Pose-Driven, Multi-Band Blending architecture** capable of generating seamless orthomosaics in real-time.

By separating localization from perception, the system successfully resolves the "Water Problem"—the situation in which visual trackers fail over textureless surfaces. We accomplish a balance between geometric consistency and visual coherence by using Laplacian pyramid blending and direct georeferencing. Although there are still issues with sensor noise and terrain relief, the system offers instantaneous, useful geospatial data. It provides an essential tool for quick flood assessment and search-and-rescue operations by converting the UAV from a passive recording device into an active, intelligent mapping agent.

REFERENCES

[1] Y. Li, X. Hu, H. Zhao, and Y. Wang, "Map2DFusion: Real-time incremental UAV image mosaicing based on monocular SLAM," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Daejeon, South Korea, 2016, pp. 5267–5274.

[2] N. Snavely, S. M. Seitz, and R. Szeliski, "Photo tourism: exploring photo collections in 3D," in *ACM SIGGRAPH*, 2006, pp. 835–846.

[3] J. L. Schönberger and J.-M. Frahm, "Structure-from-Motion Revisited," in *CVPR*, 2016.

[4] OpenDroneMap Community, "OpenDroneMap: The Missing Guide," 2023. [Online]. Available: https://opendronemap.org/

[5] Agisoft LLC, "Agisoft Metashape User Guide," 2022. [Online]. Available: https://www.agisoft.com/

[6] Pix4D, "Pix4D Mapper Technical Documentation," 2022.

[7] C. Campos et al., "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multi-Map SLAM," *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.

[8] T. Qin, P. Li, and S. Shen, "VINS-Mono," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.

[9] J. Engel, V. Koltun, and D. Cremers, "Direct Sparse Odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2018.

[10] J. Engel, T. Schöps, and D. Cremers, "LSD-SLAM: Large-Scale Direct Monocular SLAM," in *ECCV*, 2014.

[11] P. Burt and E. Adelson, "The Laplacian Pyramid as a Compact Image Code," *IEEE Trans. Communications*, vol. 31, no. 4, pp. 532–540, 1983.

[12] R. Szeliski, *Computer Vision: Algorithms and Applications*. Springer, 2010.

[13] V. Kwatra et al., "Graphcut Textures: Image and Video Synthesis Using Graph Cuts," *ACM SIGGRAPH*, 2003.

[14] M. Brown and D. G. Lowe, "Automatic Panoramic Image Stitching using Invariant Features," *IJCV*, vol. 74, no. 1, pp. 59–73, 2007.

[15] M. Milgram, "Computer Mosaicing," Technical Report, NASA, 2004.

[16] E. Honkavaara et al., "Georeferencing and Orthoimage Processing of Airborne Photogrammetric Imagery," *Photogrammetric Journal of Finland*, 2006.

[17] D. Turner, A. Lucieer, and C. Watson, "Development of an Unmanned Aerial Vehicle for Hyper-Resolution Vineyard Mapping," *Remote Sensing*, 2011.

[18] Z. Li, J. Bethel, and C. McGlone, *Manual of Photogrammetry*, 5th ed. ASPRS, 2008. (Direct georeferencing chapter)

[19] M. Erdelj, E. Natalizio, K. R. Chowdhury, and I. F. Akyildiz, "Help from the sky: Leveraging UAVs for disaster management," *IEEE Pervasive Computing*, vol. 16, no. 1, pp. 24–32, 2017.

[20] S. E. Joyce et al., "Satellite Remote Sensing for Natural Hazards," *Progress in Physical Geography*, vol. 33, no. 2, 2009.

[21] R. Feng, Q. Wu, and Y. Shi, "UAV-Based Flood Monitoring: A Comprehensive Review," *Remote Sensing*, vol. 13, no. 21, 2021.

[22] S. Hayat, E. Yanmaz, and R. Muzaffar, "Survey on Unmanned Aerial Vehicle Networks for Civil Applications," *IEEE Communications Surveys & Tutorials*, 2016.

[23] H. Almadhoun et al., "A Survey on Coverage Path Planning for UAVs," *Drones*, vol. 3, no. 4, 2019.

[24] L. C. Carrillo-Arce et al., "Cooperative Stereo SLAM for Multi-UAV Systems," in *ICRA*, 2013.

[25] A. Fotouhi et al., "Survey on UAV Cellular Communications: Practical Challenges and Applications," *IEEE Communications Surveys*, 2019.

[26] S. Negahdaripour, "On Phase-Based Methods for Motion Estimation and Image Mosaicing," *CVIU*, 1998.

[27] B. R. Ferrer et al., "Real-Time Aerial Video Mosaicking," *IEEE Trans. Geoscience and Remote Sensing*, 2017.

[28] D. DeTone, T. Malisiewicz, and A. Rabinovich, "Deep Image Homography Estimation," in *CVPR Workshops*, 2016.

[29] X. Xie et al., "SegFormer: Simple and Efficient Design for Semantic Segmentation," *NeurIPS*, 2021.

[30] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *MICCAI*, 2015.

[31] R. Murphy, *Disaster Robotics*. MIT Press, 2014.

[32] J. Chen et al., "Emergency Communication Using UAV Swarms," *IEEE Access*, 2019.

[33] Y. Li, "NPU Drone Map Dataset," Northwestern Polytechnical University, 2016. Available: https://github.com/Yuliang-Li/NPU-Drone-Map-Dataset